Lab 7 — Deploy a Container with Azure Container Instances

In this lab you run a public Docker container image on Azure Container Instances (ACI) with a single command, browse the running app on a public URL, and inspect its logs — all without provisioning any VM. This maps to the AZ-900 skill area **Describe Azure architecture and services** — container services and serverless-style compute.

Run in Azure Cloud Shell — https://shell.azure.com

Step 1 — Open Cloud Shell and ensure the resource group exists

Open https://shell.azure.com, choose **Bash**, then run:

```
az group create --name rg-az900-labs --location southeastasia
```

This is idempotent — if `rg-az900-labs` already exists from an earlier lab, the command simply returns it.

Step 2 — Create the container instance

```
az container create \
  --resource-group rg-az900-labs \
  --name aci-az900-hello \
  --image mcr.microsoft.com/azuredocs/aci-helloworld \
  --dns-name-label az900-<yourinitials><random> \
  --ports 80 \
  --os-type Linux \
  --cpu 1 \
  --memory 1
```

`--image` pulls Microsoft's sample hello-world web app from the Microsoft Container Registry; `--dns-name-label` must be unique within the region because it becomes part of a public FQDN; `--cpu 1 --memory 1` requests 1 vCPU and 1 GB RAM. The command takes about a minute and returns JSON with `"provisioningState": "Succeeded"`.

Note how fast this is compared to a VM — no OS to deploy, just a container image to pull and start.

Step 3 — Get the public FQDN

```
az container show \
  --resource-group rg-az900-labs \
  --name aci-az900-hello \
```

```
  --query ipAddress.fqdn \
  --output tsv
```

`--query` uses JMESPath to extract just the fully qualified domain name, e.g. `az900-tan4821.southeastasia.azurecontainer.io`.

Step 4 — Browse the running container

Open a browser tab and go to `http://<the-fqdn-from-step-3>`.

You should see the blue **"Welcome to Azure Container Instances!"** page. The container is serving real HTTP traffic on a public IP and DNS name that Azure provisioned for you.

Step 5 — View the container logs

```
az container logs \
  --resource-group rg-az900-labs \
  --name aci-az900-hello
```

You see the Node.js app's stdout, including a log line for the HTTP GET request your browser just made. Container logs are the primary troubleshooting tool for ACI — there is no server to SSH into.

Step 6 — Restart the container group

```
az container restart \
  --resource-group rg-az900-labs \
  --name aci-az900-hello
```

Refresh the browser tab — the page is back within seconds. Compare this to a VM reboot, which typically takes minutes: containers restart in seconds because there is no operating system to boot, only a process.

Step 7 — Compare containers vs VMs vs PaaS

No commands here — check your understanding against this table:

	Virtual machine (Lab 4/5)	Container (this lab)	App Service PaaS (Lab 6)
You manage	OS, patching, runtime, app	Container image, app	App code only

	Virtual machine (Lab 4/5)	Container (this lab)	App Service PaaS (Lab 6)
Startup time	Minutes	Seconds	Seconds (already-running platform)
Density/efficiency	1 OS per VM	Many containers share one OS kernel	Multi-tenant platform
Isolation	Strong (hardware-virtualised)	Process-level	Platform-managed
Best for	Lift-and-shift, full OS control	Portable microservices, burst jobs	Standard web apps and APIs

Containers are lightweight virtualisation: they virtualise the OS rather than the hardware, which is why they start faster and pack more densely than VMs.

Step 8 — Delete the container group

```
az container delete \
  --resource-group rg-az900-labs \
  --name aci-az900-hello \
  --yes
```

ACI bills per second while the container group exists, so deleting it immediately stops all charges.

Clean up

Step 8 already removed the only resource this lab created. Verify nothing is left:

```
az container list --resource-group rg-az900-labs --output table
```

The output should be empty. Keep the resource group `rg-az900-labs` itself — it is reused until Lab 16.

What you learned

- **Azure Container Instances** runs containers on demand with no VMs or orchestrator to manage — the fastest way to run a container in Azure.
- Containers start in **seconds** because they share the host OS kernel instead of booting their own, giving higher density than VMs.
- A `--dns-name-label` gives a container group a public FQDN of the form `<label>.<region>.azurecontainer.io`.

- `az container logs` replaces SSH for troubleshooting — you observe the app, not a server.
- Choosing between **IaaS (VMs), containers, and PaaS** is about how much of the stack you want to manage — a core AZ-900 exam theme.

Files in this lab folder

File	What it is
`deploy-container.sh`	Creates the container group and prints its FQDN: `bash deploy-container.sh <dns-label>`.
`container-group.yaml`	The same container group as declarative YAML — deploy with `az container create --file`.

The scripts are optional conveniences — the lab steps above teach the same thing command by command. Upload a script to Cloud Shell (**Manage files > Upload**), or clone this repo there, then run it with `bash <script-name>`.